Deployement

Install Docker and other components.

1. Set up Docker's `apt` repository.

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/g
pg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$V
ERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

2. Install the Docker packages.

```
sudo apt install docker-ce docker-ce-cli containerd.io doc
ker-buildx-plugin docker-compose-plugin

sudo systemctl status docker
```

3. Add your user to the `docker` group and run the following command to activate the changes to groups:

```
sudo usermod -aG docker $USER
newgrp docker
```

```
git clone https://github.com/grafana/quickpizza.git
```

```
cd quickpizza/
```

App Setup

Start the containers in monolithic mode

```
docker compose -f compose.grafana-local-stack.monolithic.yaml
up -d
```

Output:

```
[+] up 105/105
 ✔ Image grafana/tempo:2.9.0      Pulled
24.0s
 ✔ Image grafana/alloy:v1.11.3    Pulled
38.2s
 ✔ Image postgres:18.1            Pulled
39.2s
 ✔ Image prom/prometheus:v3.2.1   Pulled
34.1s
 ✔ Image ghcr.io/grafana/quickpizza-local:0.15.14 Pulled
15.6s
 ✔ Image grafana/grafana:12.2.0   Pulled
44.2s
 ✔ Image grafana/pyroscope:1.14.0 Pulled
24.8s
```

```
 ✔ Image grafana/loki:3.4.3       Pulled
23.7s
 ✔ Network quickpizza_default     Created
0.1s
 ✔ Volume quickpizza_postgres_data Created
0.0s
 ✔ Container prometheus           Created
1.5s
 ✔ Container quickpizza-db        Healthy
13.8s
 ✔ Container loki                 Created
1.5s
 ✔ Container grafana              Created
1.5s
 ✔ Container pyroscope            Created
1.5s
 ✔ Container quickpizza           Created
0.1s
 ✔ Container tempo                Created
0.1s
 ✔ Container alloy                Created
```

Do `docker ps` to make sure all the containers are running.

```
docker ps


CONTAINER ID   IMAGE                                          COM
MAND                  CREATED         STATUS
PORTS
NAMES
db22b3c8e934   grafana/alloy:v1.11.3                          "/b
in/alloy run /con…"   9 minutes ago   Up 9 minutes
0.0.0.0:4317-4318->4317-4318/tcp, [::]:4317-4318->4317-4318/t
cp, 0.0.0.0:12345->12345/tcp, [::]:12345->12345/tcp   alloy
7111d15a7f40   grafana/tempo:2.9.0                            "/t
```

```
empo -config.file…"   9 minutes ago   Up 9 minutes
0.0.0.0:3200->3200/tcp, [::]:3200->3200/tcp
tempo
30ec8d4f0847   ghcr.io/grafana/quickpizza-local:0.15.14   "/b
in/quickpizza"        9 minutes ago   Up 9 minutes
0.0.0.0:3333->3333/tcp, [::]:3333->3333/tcp, 3334-3335/tcp
quickpizza
24eaf20df2d3   postgres:18.1                               "do
cker-entrypoint.s…"   9 minutes ago   Up 9 minutes (healthy)
0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
quickpizza-db
02a1ebb8f7d3   grafana/pyroscope:1.14.0                    "/u
sr/bin/pyroscope …"   9 minutes ago   Up 9 minutes
0.0.0.0:4040->4040/tcp, [::]:4040->4040/tcp
pyroscope
8da16d85f048   grafana/loki:3.4.3                          "/u
sr/bin/loki -conf…"   9 minutes ago   Up 9 minutes
0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
loki
6a60c98dda4e   prom/prometheus:v3.2.1                      "/b
in/prometheus --w…"   9 minutes ago   Up 9 minutes
0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
prometheus
7fc6738d0c29   grafana/grafana:12.2.0                      "/r
un.sh"                9 minutes ago   Up 9 minutes
0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
```

Actual Port Mapping (from docker ps)

Service	Port
Grafana	3000
QuickPizza App	3333
Prometheus	9090
Loki	3100

Service	Port
Tempo	3200
Pyroscope	4040
Postgres	5432
Alloy	12345 / 4317 / 4318

So access everything like this:

✔️ Grafana

👉 http://<EC2-IP>:3000

✔️ QuickPizza App

👉 http://<EC2-IP>:3333

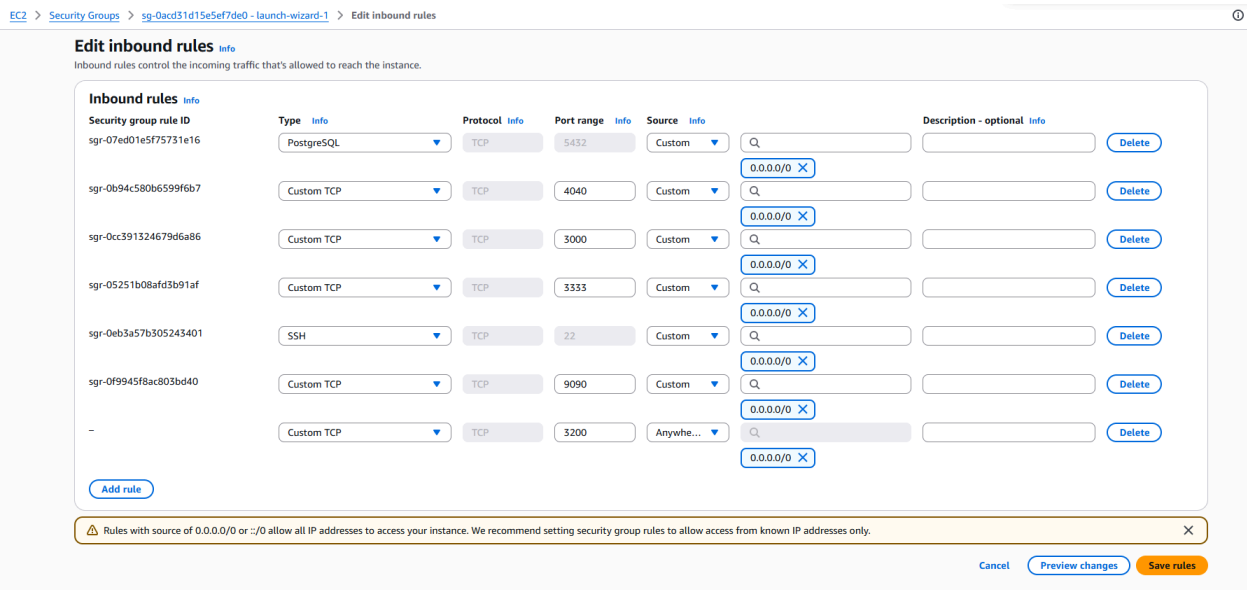
✔️ Prometheus

👉 http://<EC2-IP>:9090

✔️ Tempo

👉 http://<EC2-IP>:3200

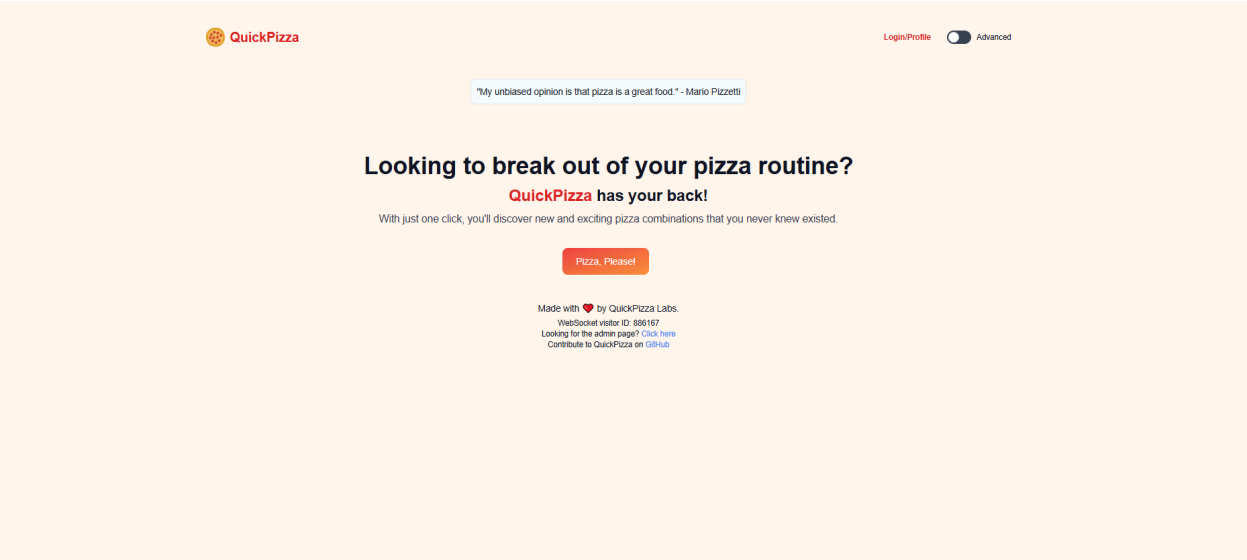
Make sure to open the respective ports in the SG.

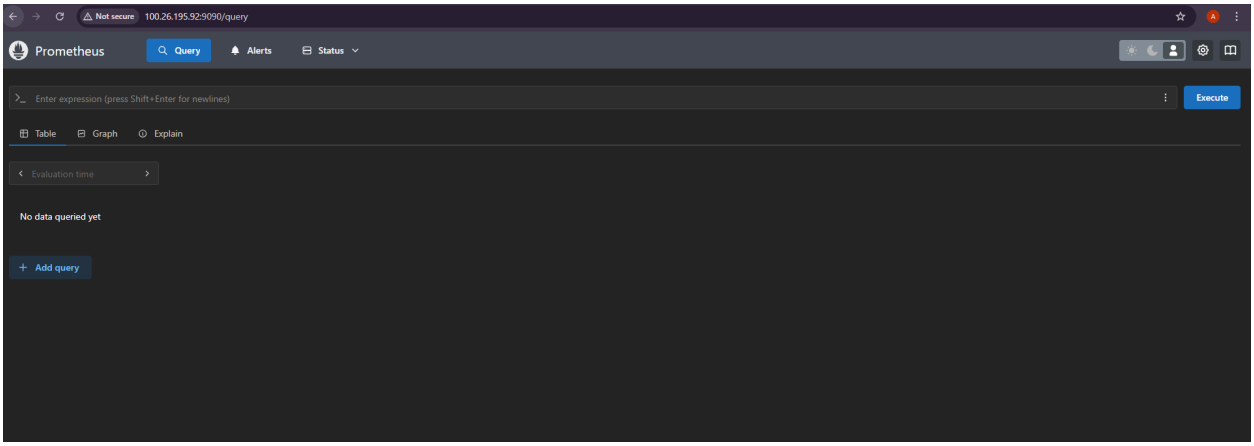


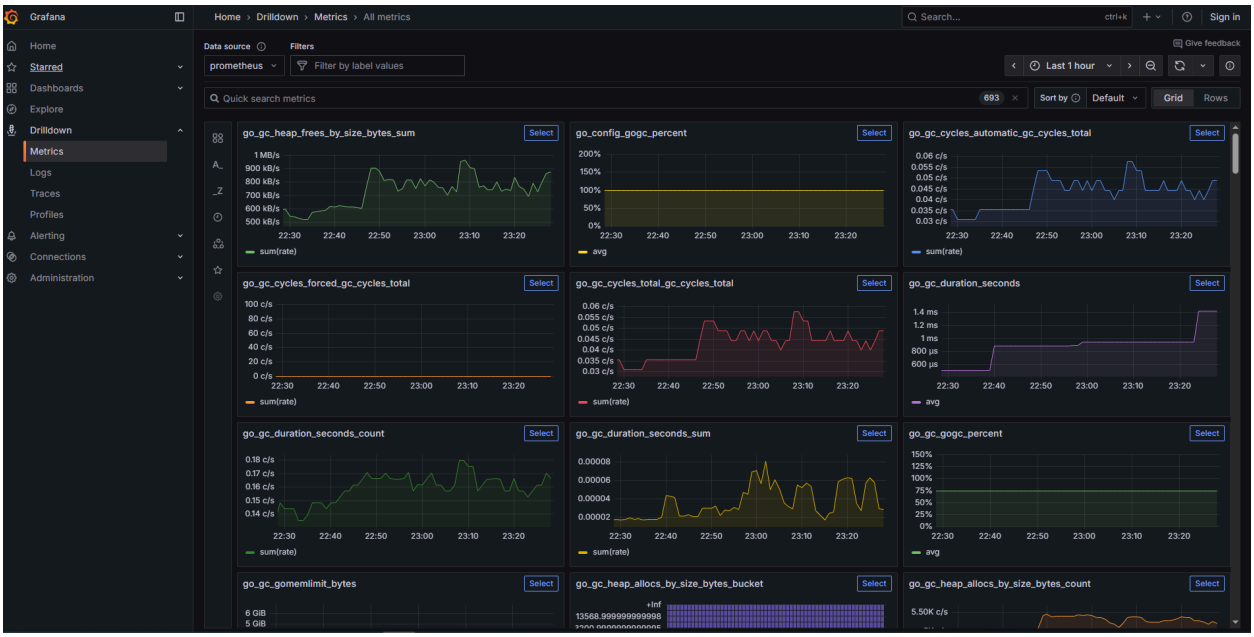
Then access the App and grafana UI using right endpoint or localhost if deploying locally.

grafana- `http://100.26.195.92:3000/` OR `localhost:3000`

App - `http://100.26.195.92:3333/` OR `localhost:3333`







Complete Flow

1. User Opens the QuickPizza App (Port 3333)

A user visits:

```
http://<EC2-IP>:3333
```

Then click around — order pizza, view menu, etc.

These actions generate:

- Metrics
- Logs
- Traces

inside the application.

---

2. The App Sends Signals to Alloy (the Collector Agent)

Think of **Alloy** as a **traffic controller** for all observability data.

It receives 4 kinds of signals:

✔️ Metrics

(e.g., request rate, error rate, latency)

✔️ Logs

(e.g., “Order created”, errors, exceptions)

✔️ Traces

(showing each request’s journey)

---

3. Alloy Routes Data to the Right Destinations

Alloy acts like a **dispatcher** and sends each type of data to the correct storage system:

Data Type	Goes To	Why
Metrics	Prometheus	For graphs, dashboard queries
Logs	Loki	For searching & filtering
Traces	Tempo	For detailed request journey

So Alloy → *forwards data* → Prometheus/Loki/Tempo

---

4. Storage Components Save the Data

Each backend stores one type of information:

🟦 Prometheus = Time series DB for metrics

Every 15s, Prometheus “pulls” metric data from Alloy.

🟧 Loki = Log storage

Stores raw logs grouped by labels (app, environment, etc.).

🟪 Tempo = Trace storage

Stores distributed traces that capture how each request moved through the app.

---

5. Grafana Reads From All These Backends

Grafana is the **visual dashboard** for EVERYTHING.

It connects to:

- Prometheus → metrics
- Loki → logs
- Tempo → traces

Grafana doesn't store data — it *displays* the data stored in other components.

---

6. You View Everything in Grafana (Port 3000)

Inside Grafana you can:

View dashboards

Latency, CPU, request rates, error percentages.

Explore logs

See real-time logs filtered by app, severity, trace ID.

See traces

Follow a single pizza order request from start → finish.

Open flamegraphs

Understand CPU hotspots and bottlenecks.

This is where **observability comes alive**.

Simple Story Version

1. **User uses the app.**
2. App generates metrics, logs, traces, profiles.
3. **Alloy** collects everything.
4. Alloy sends:
    - Metrics → Prometheus
    - Logs → Loki

- Traces → Tempo

5. **Grafana** displays all this data beautifully.
6. You open Grafana → investigate → solve problems easily.

OTLP (OpenTelemetry Protocol) — explained clearly & practically

One-line definition

OTLP is the standard wire protocol used by OpenTelemetry to send telemetry data (traces, metrics, logs, profiles) from applications to a collector (like Alloy).